

VEHICLE PARKING SYSTEM

1. Abstract

The Vehicle Parking System is a C-based application designed to manage parking operations efficiently using file handling. The system allows users to park vehicles, view parked vehicles, and calculate parking charges based on vehicle type and duration. Data is stored persistently in a file, ensuring that records are maintained even after program termination.

2. Introduction

Managing parking manually can lead to inefficiencies such as data loss, incorrect billing, and difficulty in tracking vehicles. This system provides a simple digital solution using file handling in C to store and retrieve parking data.

The system ensures accurate billing and easy retrieval of vehicle information.

3. Objectives

- To manage parking records digitally
 - To implement file handling in C
 - To calculate parking charges based on vehicle type
 - To provide a simple menu-driven interface
-

✓4. System Requirements

4.1 Hardware Requirements

- Processor: Intel i3 or above
- RAM: 2 GB minimum
- Storage: 100 MB free space

4.2 Software Requirements

- Language: C
- Compiler: GCC / Turbo C / Code::Blocks

- OS: Windows / Linux
-

5. System Architecture

Flow:

User → Menu → Function Call → File Operations → Output

- Data is stored in **parking.dat**
 - Uses **binary file operations (fwrite, fread)**
-

6. Data Structure Used

Structure: Vehicle

```
struct Vehicle {  
    int number;  
    char type[10];  
    int hours;  
};
```

7. Functional Modules

7.1 Park Vehicle

- Takes input (number, type, hours)
 - Stores data in file using fwrite()
-

7.2 Display Vehicles

- Reads file using fread()
 - Displays all parked vehicles
-

7.3 Calculate Parking Charge

- Searches vehicle by number
- Applies pricing logic:

- Bike → ₹10/hour
 - Car → ₹20/hour
 - Truck → ₹30/hour
-

8. Algorithm

General Algorithm

1. Start
 2. Display menu
 3. Take user choice
 4. Call corresponding function
 5. Perform file operation
 6. Repeat until exit
-

Algorithm: Park Vehicle

1. Open file in append mode
 2. Input vehicle details
 3. Write data using fwrite()
 4. Close file
-

Algorithm: Calculate Charge

1. Open file in read mode
 2. Input vehicle number
 3. Search record
 4. Apply charge formula
 5. Display result
-

9. Billing Logic

- Bike → Hours × 10

- Car $\rightarrow$ Hours $\times$ 20
 - Truck $\rightarrow$ Hours $\times$ 30
-

10. Complexity Analysis

- Insertion: $O(1)$
 - Search: $O(n)$
 - Display: $O(n)$
 - Space: $O(n)$ (file storage)
-

11. Sample Output

--- VEHICLE PARKING SYSTEM ---

1. Park Vehicle
 2. Display Parked Vehicles
 3. Calculate Parking Charge
 4. Exit
-

12. Advantages

- Persistent data storage
 - Simple and efficient
 - Easy to implement
 - Accurate billing
-

13. Limitations

- No delete/update feature
- No GUI

- Limited validation
-

14. Applications

- Parking lots
 - Malls
 - Offices
 - Toll systems
-

15. Future Scope

- Add database system
 - GUI development
 - Online booking system
 - Barcode/RFID integration
-

16. Conclusion

The Vehicle Parking System effectively demonstrates file handling in C. It provides a simple yet efficient solution for managing parking data and calculating charges. The system can be extended with advanced features for real-world applications.

17. Appendix: Source Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Vehicle {
```

```
    int number;
```

```
char type[10];  
int hours;  
};  
  
void parkVehicle() {  
    struct Vehicle v;  
    FILE *fp = fopen("parking.dat", "ab");  
  
    if (fp == NULL) {  
        printf("File error\n");  
        return;  
    }  
  
    printf("Enter vehicle number: ");  
    scanf("%d", &v.number);  
  
    printf("Enter vehicle type (Bike/Car/Truck): ");  
    scanf("%s", v.type);  
  
    printf("Enter parking hours: ");  
    scanf("%d", &v.hours);  
  
    fwrite(&v, sizeof(v), 1, fp);  
    fclose(fp);  
  
    printf("Vehicle parked successfully.\n");  
}
```

```

void displayVehicles() {
    struct Vehicle v;
    FILE *fp = fopen("parking.dat", "rb");

    if (fp == NULL) {
        printf("No vehicles parked yet.\n");
        return;
    }

    printf("\nVehicle No\tType\tHours\n");
    printf("-----\n");

    while (fread(&v, sizeof(v), 1, fp)) {
        printf("%d\t\t%s\t%d\n", v.number, v.type, v.hours);
    }

    fclose(fp);
}

```

```

void calculateCharge() {
    struct Vehicle v;
    int num, charge = 0;
    FILE *fp = fopen("parking.dat", "rb");

    if (fp == NULL) {
        printf("No records found.\n");
        return;
    }
}

```

```
printf("Enter vehicle number: ");
```

```
scanf("%d", &num);
```

```
while (fread(&v, sizeof(v), 1, fp)) {
```

```
    if (v.number == num) {
```

```
        if (strcmp(v.type, "Bike") == 0)
```

```
            charge = v.hours * 10;
```

```
        else if (strcmp(v.type, "Car") == 0)
```

```
            charge = v.hours * 20;
```

```
        else if (strcmp(v.type, "Truck") == 0)
```

```
            charge = v.hours * 30;
```

```
        printf("Total parking charge = Rs.%d\n", charge);
```

```
        fclose(fp);
```

```
        return;
```

```
    }
```

```
}
```

```
printf("Vehicle not found.\n");
```

```
fclose(fp);
```

```
}
```

```
int main() {
```

```
    int choice;
```

```
    while (1) {
```

```
        printf("\n--- VEHICLE PARKING SYSTEM ---\n");
```



```
printf("1. Park Vehicle\n");  
printf("2. Display Parked Vehicles\n");  
printf("3. Calculate Parking Charge\n");  
printf("4. Exit\n");  
printf("Enter choice: ");  
  
scanf("%d", &choice);  
  
switch (choice) {  
    case 1: parkVehicle(); break;  
    case 2: displayVehicles(); break;  
    case 3: calculateCharge(); break;  
    case 4: exit(0);  
    default: printf("Invalid choice\n");  
}  
}  
}
```